

Ağlara Bakış Açısı

CCNA yolculuğuna başlarken ilk yapmamız gereken şey şapkanı değiştirmek. Bugüne kadar sen bir ağ **kullanıcısıydın**; yani internete giren, video izleyen kişiydin. Ama artık bu ağı inşa eden **Network Engineer** (Ağ Mühendisi) olma yolundasın.

Ağ dediğimiz şey, senin evde kullandığın internet bağlantısına çok benzer temeller üzerine kuruludur. İki ana ev senaryosu ile başlayalım:

Home Kullanıcısı Bakış Açısı

Evde yüksek hızlı internete bağlanırken genelde iki teknoloji karşımıza çıkar.

A. Cable Internet

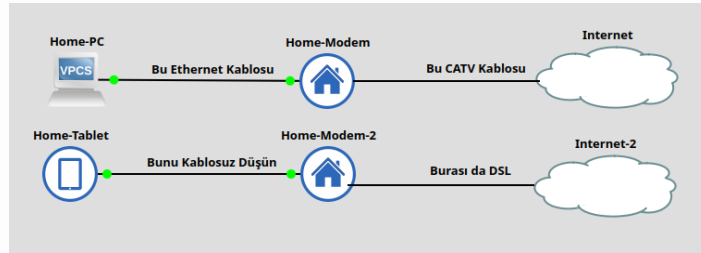
Bu senaryo genellikle TV altyapısını kullanır.

- **Bağlantı Akışı:** Bilgisayarını bir **Ethernet cable** ile **Cable Modem**'e bağlarsın.
- **Duvar Bağlantısı:** Modem ise duvardaki **CATV** (Cable TV) çıkışına bağlanır.
- **Kablo Tipi:** Duvardan modeme gelen o yuvarlak kabloya **Coaxial Cable** diyoruz (Eski anten kabloları).
- **Avantajı:** Sürekli açıktır. Oturup hemen mail atabilir, sörf yapabilirsin.

B. DSL (Digital Subscriber Line)

Bu senaryo telefon hattı altyapısını kullanır.

- **Bağlantı Akışı:** Tablet veya telefon kullanıyorsan kabloyla uğraşmazsın. **Wireless LAN** (veya yaygın adıyla **Wi-Fi**) kullanırsın.
- **Cihaz:** Burada merkezde bir **Router** vardır.
- **Teknoloji:** Router, internet ile konuşmak için **DSL** teknolojisini kullanır.



image

Burası için basit bir “Home Network” topolojisi kuracağız. Bir tarafta PC (Ethernet ile bağlı), diğer tarafta Tablet (Wi-Fi ile bağlı) ve bunları internete çıkaran bir Router simüle edeceğiz.

Adım 1: Topoloji Kurulumu

GNS3’te “Cable Modem” diye özel bir cihaz arama, bulamazsın. Çünkü ticari modemler aslında **Router**’ların özelleşmiş (ve ev için biraz kısıtlanmış) halleridir. Biz bir Router alıp ona modem kostümü giydireceğiz.

1. **Modem Rolü:** Sahneye bir Cisco Router (ben c7200 kullanıyorum) sürüküle.
 - *Görsel tiyo vereyim:* Cihaza sağ tıkla -> **Change Symbol** de ve arama kutusuna “house” veya “modem” yazıp o ikonu seç. (Maksat gözümüze gerçekçi gelsin).
 - **Adımı Değiştir:** Home-Modem yap.
2. **PC Rolü:** Sahneye bir **VPCS** (Virtual PC) sürüküle.

- **Adını Değiştir:** Home-PC yap.

Adım 2: Kablolama

PC'mizi internete çıkarmak için fiziksel bağlantıyı yapalım, **Ethernet Cable**'ı şimdi takıyoruz.

- Home-PC (Ethernet0) portunu —> Home-Modem (FastEthernet0/0) portuna bağla.

Bu işlem, senin gerçek hayatta PC kasasının arkasına taktığın ve “kık” sesini duyduğun o kablo işleminin aynısıdır.

Adım 3: Yapılandırma

Kabloları taktık, cihazların ışıkları yandı ama henüz birbirlerini tanımıyorlar. Onlara birer IP Adresi vermeliyiz.

A. Modem Ayarları

Modem, senin ev ağının patronudur, çıkış kapısıdır. Teknik dilde buna **Default Gateway** denir.

```
# GNS3'te Home-Modem'e sağ tıkla -> Console
# Aşağıdaki komutları gir:

enable                ! Yönetici moduna geç
configure terminal    ! Ayar moduna gir

# PC'nin bağlı olduğu Interface'i seçiyoruz
interface FastEthernet0/0

# Kapıya IP veriyoruz (Bu senin ağının çıkış kapısı olacak yani gateway)
ip address 192.168.1.1 255.255.255.0

# Cisco'da güvenlik gereği portlar kapalı gelir, açalım hemen.
no shutdown

exit
```

B. PC Ayarları

PC'ye kim olduğunu ve çıkış kapısının neresi olduğunu söylememiz lazım.

```
# GNS3'te Home-PC'ye sağ tıkla -> Console

# Komut formatı: ip [PC_IP] [Subnet_Mask] [Gateway_IP]
ip 192.168.1.10 255.255.255.0 192.168.1.1
```

- **192.168.1.10:** PC'nin kendi adı (Ip Adresi).
- **192.168.1.1:** PC'nin “İnternete çıkarken kime sorayım?” dediği adres (Gateway/Modem).

Adım 4: Test

Bakalım kablolar çalışıyor mu, elektrik akıyor mu? PC'den modeme bir **Ping** gönderelim.

```
# Home-PC konsolunda şu komutu yaz:
ping 192.168.1.1

Home-PC> ping 192.168.1.1

84 bytes from 192.168.1.1 icmp_seq=1 ttl=255 time=29.205 ms
84 bytes from 192.168.1.1 icmp_seq=2 ttl=255 time=6.212 ms
84 bytes from 192.168.1.1 icmp_seq=3 ttl=255 time=6.703 ms
84 bytes from 192.168.1.1 icmp_seq=4 ttl=255 time=5.641 ms
84 bytes from 192.168.1.1 icmp_seq=5 ttl=255 time=6.364 ms
```

İlk ağ bağlantısını canlandırdık. Veri paketleri PC’den çıktı, kabloyu gezdi, Modemin işlemcisine girdi ve “Selamını aldım” cevabıyla geri döndü.

Enterprise vs. SOHO Networks

Evdeki ağ ile koca bir şirketin ağı aslında “teknoloji” olarak birbirine çok benzer, sadece ölçekleri ve amaçları farklıdır. Burada iki kritik terim öğreniyoruz:

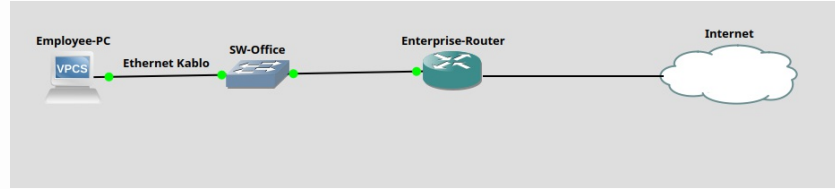
- **Enterprise Network (Kurumsal Ağ):** Büyük bir şirket tarafından, çalışanların birbiriyle iletişim kurması için inşa edilen dev ağlardır.
- **SOHO (Small Office/Home Office):** Evde iş için kurduğunuz veya küçük ofislerde kullanılan ufak ağlardır.

Benzerlik Nerede?

Bir **Enterprise Network** kullanıcısı olsan bile, deneyimin evdekine çok benzer:

1. Ofiste masana oturduğunda PC’ni duvardaki prize yine bir **Ethernet cable** ile bağlarsın.
2. Toplantı odasına gittiğinde laptopunla **Wireless LAN** (Wi-Fi) kullanırsın.

Fiziksel olarak yaptığın eylem aynıdır, ama arka planda (duvarın arkasında) dönen işler Enterprise ağlarda çok daha karmaşıktır.



Burası için “Enterprise” simülasyonu yapacağız. Evdeki basit modem yerine, profesyonel **Switch** ve **Router**’lar koyarak kurumsal bir yapının minyatürünü çizeceğiz. Zaten Home Network hallettik, modemimizi kurduk. Şimdi seviye atlıyoruz. Artık tek bir oda değil, koca bir ofis binasını yöneten mühendis kafasına geçiyoruz. Burası **Enterprise Network** dünyası.

Buradaki temel fark şu: Evde kabloyu direkt modeme takarsın. Ama büyük bir şirkette yüzlerce çalışan var. Hepsini tek bir router’a bağlayamazsın. Araya trafiği düzenleyen, çok portlu bir “trafik polisi” koymak gerekir. İşte sahneye burada **Switch** çıkıyor.

Adım 1: Topolojiyi Kur

GNS3’ü aç ve malzemeleri sahneye dizelim. Bu sefer daha “profesyonel” cihazlar kullanıyoruz.

1. **Employee-PC (VPCS):**
 - Bu, şirketteki muhasebeci Irem Hanım’ın bilgisayarını olsun.
2. **SW-Office (Ethernet Switch):**
 - Sol menüdeki “Switch” ikonunu al.
 - **Görevi:** PC’lerden gelen kabloları toplamak. Bunu binanın “Lobi”si gibi düşün. Herkes önce lobiye (Switch) iner, oradan dışarı çıkar.
3. **Enterprise-Router (Router):**
 - Cisco Router’ını sürükleyin.
 - **Görevi:** Şirketin dış dünyaya (Internet/WAN) açılan kapısı.

Adım 2: Kablolama

Şimdi o meşhur **Ethernet Cable** bağlantılarını yapalım. Verinin akacağı yolu fiziksel olarak

döşüyoruz.

- **PC -> Switch:**

- Employee-PC (Ethernet0) portunu al, SW-Office'in **1. Portuna** tak.

- **Switch -> Router:**

- SW-Office'in **2. Portunu** al, Enterprise-Router'in **FastEthernet0/0** portuna tak.

Evde PC direkt Router'a girerdi. Burada PC önce Switch'e, sonra Switch Router'a gidiyor. Bu hiyerarşi, Enterprise ağların temel taşıdır.

Adım 3: Konfigürasyon

Kablolar tamam, ışıklar yandı. Şimdi cihazlara IP Adresi verip konuşturma zamanı.

A. Router Ayarları

Router, bu ağın çıkış kapısıdır (Gateway). Ona ağın patron'u olduğunu söylememiz lazım.

```
# Enterprise-Router Konsolu:
enable
configure terminal

# Switch'e bakan portu seçiyoruz
interface FastEthernet0/0

# Gateway IP veriyoruz
ip address 192.168.1.1 255.255.255.0

no shutdown

exit
```

B. Switch Ayarları

Burası ilginç: **Switch'e IP vermemize gerek yok!** Neden? Çünkü Switch (Layer 2), temel olarak "aptal" bir kutu gibi davranır (tabii ilk kurulumda). Sadece kablodan gelen elektriği diğer porta iletir. O yüzden Switch'i açman yeterli, ekstra ayar yapmana şimdilik gerek yok. Tak ve çalıştır.

C. PC Ayarları (Çalışan)

PC'ye kim olduğunu ve çıkış kapısının (Router) neresi olduğunu öğretiyoruz.

```
# Employee-PC Konsolu:
# Komut: ip [Kendi_IPsi] [Maske] [Gateway_IPsi]

ip 192.168.1.20 255.255.255.0 192.168.1.1
```

- **IP:** 192.168.1.20 (PC'nin ofisteki oda numarası)
- **Gateway:** 192.168.1.1 (Router'ın adresi)

Adım 4: Test

Her şey hazır. Şimdi PC'den bir paket yollayalım. Bu paket kablodan çıkacak, Switch'e girecek, Switch onu Router'a fırlatacak. Router "aldım" diyip geri dönecek.

```
# Employee-PC konsolunda:
ping 192.168.1.1
```

Employee-PC> ping 192.168.1.1

```
84 bytes from 192.168.1.1 icmp_seq=1 ttl=255 time=9.541 ms
84 bytes from 192.168.1.1 icmp_seq=2 ttl=255 time=5.470 ms
84 bytes from 192.168.1.1 icmp_seq=3 ttl=255 time=5.169 ms
84 bytes from 192.168.1.1 icmp_seq=4 ttl=255 time=6.010 ms
84 bytes from 192.168.1.1 icmp_seq=5 ttl=255 time=5.625 ms
```

Artık sadece evdeki modemi resetleyen biri değilsin; PC, Switch ve Router hiyerarşisine sahip gerçek bir basic **Enterprise Network** kurdun ve çalıştırdın.

Cloud Kavramı (Ufak Bir Not)

Ağ şemalarında sürekli bir **Bulut** ikonu göreceksin. Bu ikonun çok net bir anlamı vardır:

- **Anlamı:** “Ağın bu kısımdaki detaylar şu anki konumuz için önemsizdir.”
- **Mantığı:** Örneğin evden internete çıkış şemasında, internetin kendisini bir bulut olarak çizeriz. O bulutun içinde milyarlarca kablo, sunucu ve router vardır ama biz o an sadece “senin evindeki modemi” konuştuğumuz için, geri kalan karmaşayı bulutun içine gizleriz.

Bunu bir kargo gönderimi gibi düşün. Paketi şubeye verirsin (Giriş) ve arkadaşın paketi alır (Çıkış). Arada o paket hangi kamyonu bindi, hangi depoda bekledi, hangi uçağa aktarıldı bilmezsin ve ilgilenmezsin. İşte o aradaki bilinmezlik/detay **Bulut**ur.

Ağların Temel Görevi

Bazı kullanıcılar ağın nasıl çalıştığını hiç umursamaz. Sadece Instagram’a girmek, müzik dinlemek isterler. “Nasıl oluyor da bu mesaj gidiyor?” diye düşünmezler.

Ama sen **CCNA** adayı olarak bunu bilmek zorundasın. Kitabın(CCNA Official Cert) ve bu eğitimin geri kalanında öğreneceğimiz her şey tek bir amaca hizmet ediyor. Ağların görevi şudur: > (Veriyi bir cihazdan diğerine taşımak.)

Bizim işimiz; bu taşıma işlemini yapan, güvenli kılan ve hızlandıran o “**Enterprise Network**”leri nasıl inşa edeceğimizi öğrenmektir.

TCP/IP Networking Model (Oyunun Kuralları)

Ağ dünyasına girdiğinde sürekli duyacağın bir terim var: **Networking Model** (bazen **Networking Architecture** veya **Networking Blueprint** de denir).

Peki nedir bu? Teknik olarak: Ağın çalışması için gereken her şeyi tanımlayan kapsamlı dokümanlar bütünüdür. Bizim dilimizde: **Bu işin anayasasıdır.**

Bu dokümanlar iki ana şeyi belirler:

1. **Protocol:** Cihazların iletişim kurmak için izlemesi gereken **mantıksal kurallar**. (Yazılım, dil, konuşma sırası).
2. **Physical Requirements (Fiziksel Gereksinimler):** Kablonun içinden geçecek voltaj seviyesi, akım miktarı, kablo tipi gibi **donanımsal kurallar**.

Ev Planı Analojisi

Bu “Networking Model” olayını en iyi bir **Ev İnşaata Planı (Blueprint)** ile anlarsın.

Diyelim ki bir ev yapacaksın. Kafana göre tuğla örüp, rastgele kablo çekebilir misin? Belki yaparsın ama o ev muhtemelen çöker veya musluktan elektrik çarpar. Bunun yerine bir **Blueprint** (Mimari Plan) kullanırsın.

- **Neden Plan Kullanırız?**
- Evin temeli sağlam olsun, çökmesin diye.
- Tesisatçı, Elektrikçi ve Duvarcı birbirine engel olmadan çalışabilsin diye. Elektrikçi plana bakar, prizi nereye koyacağını bilir. Tesisatçı plana bakar, boruyu nereden geçireceğini bilir. Kimse kimsenin işini bozmaz.

Ağ modelindeki kurallar sayesinde; kabloyu üreten firma, switch'i üreten firma ve bilgisayarı üreten firma birbirlerinden habersiz olsalar bile uyumlu ürünler yaparlar.

Neden Buna Muhtacız?

Şöyle bir senaryo düşün: Kendi ağını kurmak istiyorsun.

- Kendi yazılımını yazdın.
- Kendi ağ kartını lehimledin.
- Kendi kablo tipini icat ettin.

Bunu yapabilir misin? Evet. Ama bu inanılmaz zordur ve senin cihazın dünyadaki başka hiçbir cihazla konuşamaz.

Bunun yerine ne yapıyoruz? Gidip marketten **Well-known Networking Model** (herkesin bildiği ağ modeli) standartlarına uyan ürünler alıyoruz. Çünkü **Vendor**'lar (Cisco, HP, Dell gibi üreticiler), ürünlerini bu Blueprint'e sadık kalarak üretirler.

Cisco marka bir Router ile, Dell marka bir Laptop, aralarında Samsung marka bir telefon olsa bile kusursuzca anlaşır. Çünkü hepsi aynı "Anayasayı" (Networking Model) okumuştur.

TCP/IP'ye Giden Tarihçe

Bugün dünya genelinde bilgisayar ağları tek bir model kullanır: **TCP/IP**. Ancak dünya her zaman bu kadar basit değildi. "Bir zamanlar", TCP/IP dahil hiçbir ağ protokolü yoktu. **Vendors** (Üreticiler/Satıcılar) ilk ağ protokollerini kendileri yarattılar ve bu protokoller **sadece o markanın** bilgisayarlarını destekliyordu.

1. Özel/Kapalı Modeller Çağı - 1980s

1970'ler ve 80'lerde, pazarın devi **IBM** firmasıydı. IBM, 1974'te **Systems Network Architecture (SNA)** adını verdiği kendi ağ modelini yayınladı. Diğer üreticiler de kendi modellerini çıkardı.

Sorun Neydi?

Eğer şirketin üç farklı markadan (Örn: IBM, DEC, HP) bilgisayar aldıysa, mühendisler üç ayrı ağ kurmak zorundaydı. Bu ağları birbirine bağlamak tam bir kabustu. Bunu şöyle düşün: Şirketteki IBM bilgisayarlar sadece Almanca, DEC bilgisayarlar sadece Japonca konuşuyor. Birbirlerini asla anlamıyorlar. Tercüman bulmak zorundasın.

[1980'ler - Parçalanmış Ağlar]

```
[ IBM Network ] <-----> (Sadece IBM Bilgisayarlar)
|
X (İletişim Yok / Zor)
|
[ DEC Network ] <-----> (Sadece DEC Bilgisayarlar)
|
X (İletişim Yok / Zor)
|
[ Other Vendor Network ] <----> (Diğer Marka Bilgisayarlar)
```

Gördüğün gibi, her marka kendi adasında yaşıyor.

2. Open Model Arayışı

Bu karmaşayı çözmek için **Vendor-neutral** (Markadan bağımsız) bir modele ihtiyaç vardı. Rekabeti artırmak ve karmaşıklığı azaltmak gerekiyordu. İki büyük rakip sahneye çıktı:

Aday 1: OSI Model

- **Yaratan:** **ISO** (International Organization for Standardization).
- **Yaklaşım:** 1970'lerin sonunda başladılar. Hedefleri çok asil ve büyüktü: Dünyadaki tüm bilgisayarları konuşuşturmak.
- **Katılımcılar:** Dünyanın teknolojik olarak gelişmiş birçok ülkesi. Çok resmi ve bürokratik bir süreç.

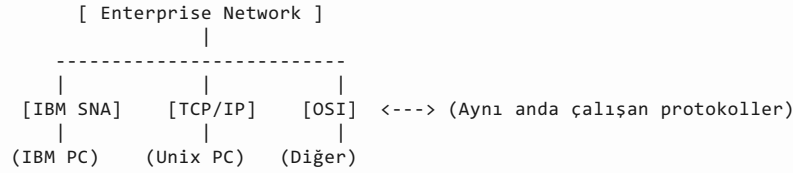
Aday 2: TCP/IP

- **Yaratan:** **U.S. Department of Defense (DoD)** (ABD Savunma Bakanlığı) sözleşmesiyle filizlendi.
- **Yaklaşım:** Daha az resmi. Üniversitelerdeki gönüllü araştırmacılar protokolleri geliştirdi.
- **Katılımcılar:** Gönüllüler, akademisyenler.

3. Geçiş Dönemi - 1990s

1990'larda şirketler ağlarına hem OSI, hem TCP/IP hem de eski özel modelleri eklemeye başladılar. Tam bir geçiş süreciydi.

[1990'lar - Karışık Ortam]



Ağlar hala karmaşık ama TCP/IP yavaş yavaş içeri sızıyor.

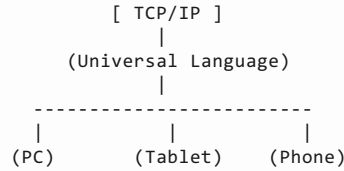
4. TCP/IP'nin Zaferi - 2000s ve Günümüz

90'ların sonuna gelindiğinde kazanan belli oldu: **TCP/IP**.

- **OSI Neden Kaybetti?** Geliştirme süreci çok yavaştı. Resmi standartlaştırma bürokrasisi yüzünden pazarın hızına yetişemedi.
- **TCP/IP Neden Kazandı?** Gönüllüler tarafından geliştirilen bu model, pratikti, çalışıyordu ve hızla yayıldı.

Bugün (21. Yüzyıl), özel modeller neredeyse tamamen terk edildi.

[Günümüz - Tek Dil]



Artık marka ne olursa olsun, herkes TCP/IP konuşuyor.

Önemli Bir Not: OSI Öldü mü?

Burada çok kritik bir noktaya değiniyoruz. “Hiç OSI protokollerini kullanan bir bilgisayarda

çalışacak mısın?” **Muhtemelen hayır.** “Peki neden öğreniyoruz?” Çünkü **OSI Jargonu** (Terimleri) hâlâ kullanılıyor.

TCP/IP’ye Genel Bakış

TCP/IP modeli, bilgisayarların konuşmasını sağlayan devasa bir protokoller koleksiyonudur. Tek bir kural değil, kurallar bütünüdür.

1. Belgeler ve Standartlar

Bu kurallar nerede yazıyor?

- **RFC (Requests For Comments):** TCP/IP, bir protokolü tanımlamak için RFC adı verilen belgeleri kullanır. (Bunları Google’da aratıp bulabilirsin, hepsi halka açıktır).
- **Tekeri Yeniden İcat Etmem:** TCP/IP modeli çok akıllıdır. Eğer başka bir kurum bir şeyi zaten standartlaştırdıysa, TCP/IP onu baştan yazmaz, sadece ona referans verir.
- **Örnek: IEEE** (Institute of Electrical and Electronic Engineers) kurumu **Ethernet LAN** standartlarını zaten belirlemiştir. TCP/IP, RFC’lerinde Ethernet’i sıfırdan tanımlamaz, “Ethernet için IEEE standartlarına bakınız” der geçer.

2. Kutudan Çıktığı Gibi Çalışma

Yeni bir telefonu veya PC’yi kutudan çıkarırsın, kabloyu takarsın ve çalışır. Nasıl?

- **OS (İşletim Sistemi):** Windows, Linux veya Android, yazılım tarafında TCP/IP modelinin parçalarını zaten içinde barındırır.
- **Hardware (Donanım):** Bilgisayardaki **Ethernet Card** veya **Wireless LAN Card**, TCP/IP modelinin referans verdiği donanım standartlarına göre üretilmiştir.

Kısacası, donanım ve yazılım üreticileri, ürünlerini bu modele göre ürettikleri için her şey tıkır tıkır çalışır.

TCP/IP Katmanları

Bir ağ modelini anlamanın en iyi yolu, karmaşık işleri küçük parçalara bölmektir. Bu parçalara **Layer (Katman)** diyoruz. Her katmanın kendi görev alanı vardır. Aşağıda gösterilen modern **5 Katmanlı TCP/IP Modelini** görüyoruz.

[TCP/IP Networking Model]

	Application		<--- (En Üst: Uygulamalar ve Kullanıcı)
	Transport		<--- (Taşıma: Veri güvenliği ve akışı)
	Network		<--- (Ağ: Adresleme ve Rota çizme)
	Data Link		<--- (Veri Bağlantısı: Özel bağlantı kuralları)
	Physical		<--- (Fiziksel: Kablolar ve Bit’ler)

Katmanların Görev Dağılımı

1. **Physical Layer:** Bit’lerin (0 ve 1) fiziksel kablo veya hava üzerinden nasıl iletileceğine odaklanır.
2. **Data-Link Layer:** Veriyi belirli bir fiziksel bağlantı tipi üzerinden göndermeye odaklanır.
- **Örnek:** **Ethernet LAN** için kurallar farklıdır, **Wireless LAN** için farklıdır.
3. **Network Layer:** Veriyi, gönderen bilgisayardan alıp, dünyanın öbür ucundaki alıcı

bilgisayara kadar **tüm yol boyunca** teslim etmeye odaklanır.

4. **Transport & Application Layers:** Veriyi göndermek ve almak isteyen uygulamalara (Web tarayıcısı, E-posta vb.) odaklanır.

ÖNEMLİ NOT (RFC 1122):

Eski kaynaklarda veya RFC 1122’de “4 Katmanlı” bir TCP/IP modeli görebilirsin. Ancak, **Real Networking** (Gerçek hayat) ve **CCNA Sınavı** için yukarıdaki **5 Katmanlı Model** esas alınır.

Örnek Protokoller

Aşağıdaki tabloda, hangi protokolün hangi katmanda çalıştığını görebilirsin. Bu tabloyu adın gibi bileceksin, çünkü eğitimlerin geri kalanı bu tabloyu detaylandırmakla geçecek.

[TCP/IP Architecture]

TCP/IP Layer	Örnek Protocols
Application	HTTP, POP3, SMTP
Transport	TCP, UDP
Internet (Network)	IP, ICMP
Data Link & Physical	Ethernet, 802.11 (Wi-Fi)

Not: Tabloda “Internet” olarak geçen katman, 5 katmanlı modelde “Network” katmanına karşılık gelir. İkisi de kullanılır.

- Application:** Web siteleri (HTTP), E-posta (SMTP, POP3).
- Transport:** Veri taşıma yöntemleri (TCP, UDP).
- Internet/Network:** Adresleme (IP).
- Link & Physical:** Kablolü (Ethernet) ve Kablosuz (Wi-Fi) bağlantılar.

TCP/IP Application Layer (Uygulama Katmanı)

Burası TCP/IP modelinin en tepesidir ve biz kullanıcılara en yakın olan katmandır.

1. Görevi Nedir?

Burada çok önemli bir ayrım var, sakın karıştıрма:

- Application Layer**, bilgisayarında çalışan uygulamanın **kendisi değildir** (Yani Chrome, Firefox veya Outlook programının kendisi değildir - Internet olmadan bi’ HTML dosyası aç bakalım ne oluyor?).
- Görevi:** Bu uygulamaların ağa erişmek için ihtiyaç duyduğu **servisleri** tanımlamaktır.

Kısacası, Uygulama Katmanı, senin bilgisayarındaki yazılım ile ağın geri kalanı arasında bir **interface** sağlar. Garsonu düşün. Garson yemeğin kendisi değildir, aşçı da değildir. Garson, senin (Uygulama) mutfakla (Network) iletişim kurmanı sağlayan araçtır. “Bana su getir” dediğinde o isteği anlayan ve ileten protokoldür.

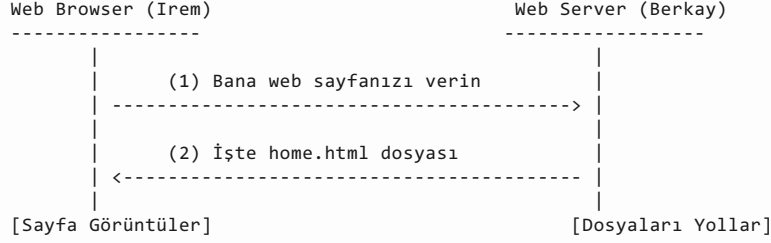
2. HTTP Overview (Bir Web Sayfası Nasıl Açılır?)

Bugün dünyanın en popüler TCP/IP uygulaması tartışmasız **Web Browser** (Tarayıcı)’dır. Hatta birçok yazılım firması artık programlarını tarayıcı üzerinden çalışacak şekilde değiştiriyor. Peki sen tarayıcıya “www.google.com” yazıp Enter’a bastığında, o sayfa sihirli bir şekilde nasıl önüne geliyor? Burada **HTTP** (Hypertext Transfer Protocol) devreye

giriyor.

Bunu bi' senaryoyla gösterelim, **Irem** (Kullanıcı) ve **Berkay** (Sunucu) üzerinden canlandıralım.

[Bir Web Sayfasını Almak İçin Temel Uygulama Mantığı]



3. Perde Arkasında Ne Oldu?

Yukarıdaki şemada basit gibi görünen işlem aslında şöyle gerçekleşir:

1. **İstek (Request):** Irem tarayıcısını açar. Tarayıcı, Berkay'ın web sunucusundan varsayılan sayfayı isteyecek şekilde ayarlanmıştır. Irem, Berkay'a "Bana ana sayfamı gönder" mesajını yollar.
2. **İşleme (Processing):** Berkay'ın sunucu yazılımı bu isteği alır. Kendi ayarlarında varsayılan sayfanın **home.html** adlı bir dosya olduğunu bilir ve diskinden bu dosyayı bulur.
3. **Yanıt (Reply):** Berkay, bu **home.html** dosyasını paketleyip Irem'e geri gönderir.
4. **Görüntüleme:** Irem dosyayı alır ve tarayıcı penceresinde sana o güzel web sayfasını gösterir.

İşte **Application Layer** (burada HTTP protokolü), Irem'in "Ver" demesiyle Berkay'ın "Al" demesi arasındaki o konuşma kurallarını belirleyen katmandır.

HTTP Protocol Mekanizması

Uç noktadaki (Endpoint) uygulamalar, yani Browser ve Server, birbirleriyle anlaşmak için **HTTP** dilini kullanır bunu öğrenmiştik.

Bu protokol, 1990'ların başında **Tim Berners-Lee** tarafından icat edildi. Amacı basitti: Tarayıcılara dosya isteme (**Request**), sunuculara da bu dosyayı gönderme (**Reply**) yeteneği kazandırmak.

Ufak Bir Not (URL/URI):

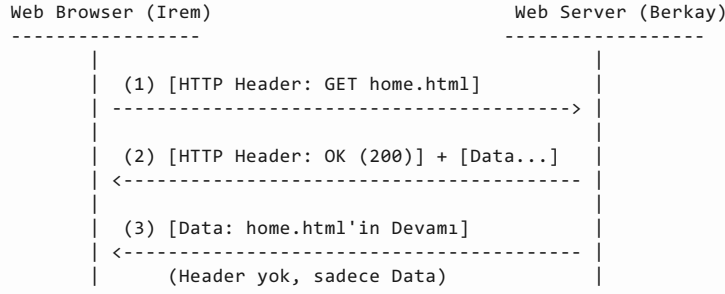
Web adreslerinin (URL veya URI denir) başındaki **http** veya **https** ibaresi, tarayıcıya "Bak bu sayfayı çekerken HTTP kurallarını kullanacaksın" emrini verir.

Mekanizma: Başlıklar ve Veri

Protokollerin birbirine derdini anlatmak için kullandığı en önemli araç **Header (Başlık)** dediğimiz bilgi etiketleridir. Bir mektubun zarfı gibi düşünebilirsiniz; zarfın üstünde adres yazar (Header), içinde ise mektup (Data) vardır.

Hadi Irem ve Berkay arasındaki trafiği bu sefer teknik göz üzerinden inceleyelim.

[HTTP GET Request, Reply, ve Data Stream]



Adım Adım Neler Oluyor?

Adım 1: The Request (İstek)

Irem, Berkay'a bir mesaj atar. Bu mesajın bir **HTTP Header**'ı vardır.

- **Komut:** Header'ın içinde **GET** komutu yazar. Yani "Bana şu dosyayı getir".
- **Dosya Adı:** Genelde dosya adı belirtilir (home.html). Eğer belirtilmezse sunucu otomatik olarak varsayılan ana sayfayı anlar.

Adım 2: The Response (Yanıt ve Kodlar)

Berkay mesajı alır ve cevap verir. Bu cevabın da bir **Header**'ı vardır.

- **Return Code (Dönüş Kodu):** Berkay, işlemin sonucunu bir sayı ile bildirir.
- **200:** "Her şey yolunda, işlem **OK**."
- **404:** "Aradığın dosya bende yok (**Not Found**).". (Hani internette gördüğün o meşhur hata).
- **Data:** Mesajın devamında istenen dosyanın (**home.html**) ilk parçası bulunur.

Adım 3: Verimlilik ve Sadece Veri

Burası çok zekice. Dosya büyükse tek pakete sığmaz, parça parça gelir. Berkay dosyanın geri kalanını gönderirken **artık Header koymaz. Neden?** Çünkü zaten anlaştılar, başlık ekleyip boşuna yer kaplamaya gerek yok. Sadece saf veriyi gönderir.

TCP/IP Transport Layer (Taşıma Katmanı)

Uygulama katmanında (Application Layer) yüzlerce protokol (HTTP, SMTP, POP3 vs.) varken, bir alt katman olan **Transport Layer**'da işler daha sadedir. Burada genellikle iki ana protokol borusu öter:

1. **TCP (Transmission Control Protocol)**
2. **UDP (User Datagram Protocol)**

Servis Mantığı

Ağ modellerinde altın kural şudur: **Her katman, bir üstündeki katmana hizmet eder.**

- **Transport Layer**, bir üstündeki **Application Layer**'a hizmet sunar.
- **Nasıl?** Örneğin HTTP (Application), veriyi gönderir ve arkasına yaslanır. "Bu veri karşıya ulaştı mı, yolda kayboldu mu?" diye dert etmez. Çünkü bu derdi onun yerine **TCP** (Transport) üstlenir. TCP'nin sunduğu en büyük hizmetlerden biri **Error Recovery** (Hata Kurtarma) özelliğidir.

TCP Hata Kurtarma Temelleri

Irem ve Berkay örneğine geri dönelim. Irem, Berkay'dan home.html sayfasını istemişti.

Ama ağ dünyası tekin bir yer değildir. Kablolar kopabilir, routerlar yoğunluktan paket atabilir.

- **Senaryo:** Irem isteği gönderdi ama yolda kayboldu.
- **Sonuç:** TCP olmasaydı, Irem sonsuza kadar boş ekrana bakardı. Sayfa yüklenmezdi.

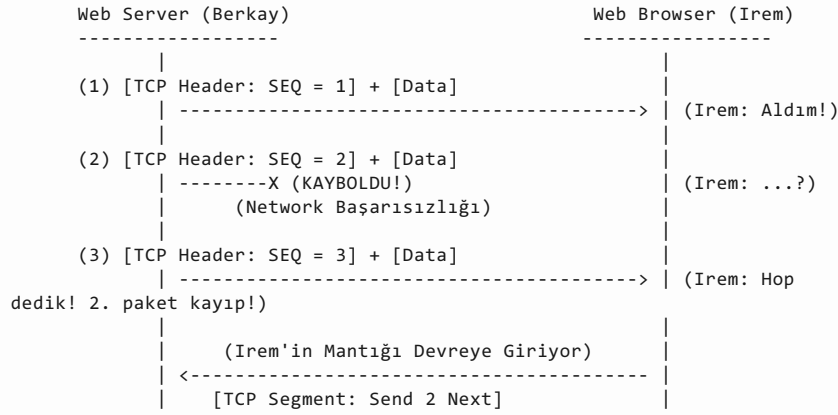
İşte bu yüzden TCP, verinin karşıya ulaştığını **garanti etmek** zorundadır. Bunu yapmak için **Acknowledgments (ACK)** ve **Sequence Numbers (SEQ)** dediğimiz bir mekanizma kullanır.

Sıralama ve Kurtarma

TCP, gönderdiği her veri parçasına (buna **Segment** diyoruz) bir sıra numarası (**Sequence Number - SEQ**) verir. Tıpkı kargoları “Koli 1, Koli 2, Koli 3” diye etiketlemek gibi.

Hadi senaryoyu canlandıralım. Berkay, Irem’e web sayfasını 3 parça halinde gönderiyor ama 2. parça yolda kayboluyor.

[TCP Hata Kurtarma İşlemi]



Adım Adım Olay Örgüsü

1. **SEQ = 1:** Berkay ilk parçayı yollar. Irem bunu alır. Her şey yolunda.
2. **SEQ = 2:** Berkay ikinci parçayı yollar. Ancak ağda bir sorun olur ve bu paket **kaybolur (Lost)**. Irem’in haberi bile yok.
3. **SEQ = 3:** Berkay üçüncü parçayı yollar. Irem bunu alır.
4. **Error Detection:** İşte zeka burada devreye girer. Irem elindeki kutulara bakar: “Elimde 1 var, 3 var... Ee 2 nerede?” Irem, aradaki parçanın gelmediğini anlar.
5. **Recovery (Kurtarma):** Irem’in TCP mantığı hemen Berkay’a bir mesaj döner: “**Hey Berkay! 1 ve 3 geldi ama 2 yok. Bana 2’yi tekrar gönder (Send 2 Next).**”

Böylece eksik parça tamamlanır ve uygulama (HTTP) hiçbir şey olmamış gibi web sayfasını bütün halinde gösterir. Bu örnekte verinin taşındığı kutuya **Segment** (Parça) adını verdiğimiz unutma. Transport katmanının veri birimi “Segment”tir.

Interactions (Etkileşim Türleri)

Ağ iletişimde iki farklı türde “konuşma” vardır. Biri kendi bilgisayarının içinde gerçekleşir, diğeri ise karşıdaki bilgisayarla.

Komşu Katman Etkileşimi (Adjacent-Layer)

Bu etkileşim, **tek bir bilgisayar üzerinde** gerçekleşir.

- **Yönü:** Dikey. Yukarıdan aşağıya veya aşağıdan yukarıya.
- **Mantığı:** Bir üst katman, işini yaptırmak için bir alt katmandan **hizmet ister**. Alt katman da bu hizmeti **sağlar**.

Örnek (Patron ve Asistan): * **HTTP (Patron - Üst Katman):** “Ben bu veriyi göndermek istiyorum ama hatayla, kayıpla uğraşamam.” der. * **TCP (Asistan - Alt Katman):** “Sen merak etme patron, paket kaybolursa tekrar isteme işi (Error Recovery) bende.” der.

Yani HTTP, hatasız iletim istiyordu, bu yüzden bir altındaki komşusu olan TCP’yi kullandı.

Aynı Katman Etkileşimi (Same-Layer)

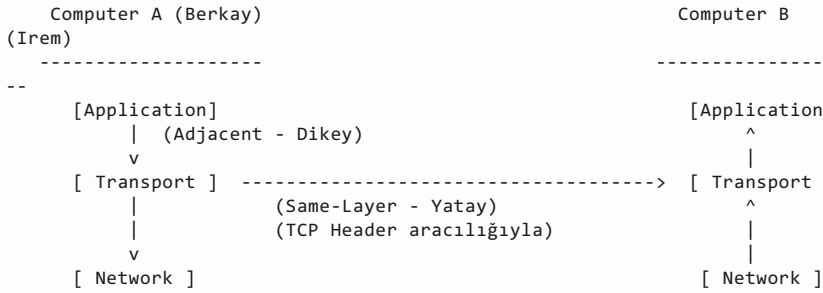
Bu etkileşim, **farklı bilgisayarlar arasında** gerçekleşir.

- **Yönü:** Yatay.
- **Mantığı:** Bir bilgisayardaki katman, karşı bilgisayardaki **kendi dengiyle** konuşur.
- **Aracı:** Bunu yapmak için **Headers** kullanılır.

Örnek (Diplomatik Mektup): Berkay’ın TCP’si bir mektup yazar (Header) ve içine “Seq No: 1” yazar. Bu notu Irem’in HTTP’si okumaz, Irem’in IP’si okumaz. Bu notu sadece ve sadece mektubun muhatabı olan **Irem’in TCP’si** okur ve anlar.

Daha iyi anlamak için bu iki kavramın tek karede özeti. Oklar kimin kiminle konuştuğunu gösteriyor:

[Etkileşim Türleri]



- **Dikey Oklar:** Komşu katman etkileşimi (Hizmet alıp verme).
- **Yatay Ok:** Aynı katman etkileşimi (Header ile haberleşme).

Özet Tablosu

Bu tabloyu bir yere mutlaka yaz, sınavda kafan karışırsa hemen bu tabloyu hatırla.

[Etkileşimler Özeti]

Kavram	Bağlam	Açıklama
Same-Layer Interaction	Farklı Bilgisayarlar	İki bilgisayar, protokoller aracılığıyla birbiriyle konuşur. Ne yapmak istediklerini Header içine yazarak birbirlerine iletirler. (Örn: TCP SEQ numaraları).
Adjacent-Layer Interaction	Tek Bilgisayar	Bir alt katman, bir üst katmana hizmet sunar. Yazılım veya donanım, bir altındaki katmandan işini yapmasını ister. (Örn: HTTP’nin TCP’den hata düzeltme istemesi).

TCP/IP Network Layer

Hatırlarsan **Application Layer**'da yüzlerce protokol vardı. **Transport Layer**'da ise TCP ve UDP başroldeydi. Ama **Network Layer**'a geldiğimizde sahne tek bir dev isme kalıyor: **IP (Internet Protocol)**. Tüm modelin adı neden **TCP/IP** biliyor musun? Çünkü en çok iş yapan iki protokolün (Transport'taki **TCP** ve Network'teki **IP**) adını birleştirip araya bir taksim (/) koymuşlar.

IP'nin iki temel süper gücü vardır:

1. **Addressing**: Herkese benzersiz bir numara vermek.
2. **Routing**: Veriyi en iyi yoldan hedefe götürmek.

IP vs. The Postal Service (Posta Servisi Analojisi)

Bu katmanı anlamak için PTT veya Kargo mantığını düşünmen yeterli.

1. Kullanıcının Gözünden

Diyelim ki iki mektup yazdın:

- Biri mahalledeki arkadaşına (Local).
- Diğeri ülkenin öbür ucundaki arkadaşına (Remote).

Zarfların üzerine adresleri yazdın, pulu yapıştırdın ve posta kutusuna attın.

Soru: Senin için bu iki mektup arasında bir fark var mı? **Cevap**: Hayır. Sen sadece kutuya atarsın ve gitmesini beklersin. Arkada tır mı kullanıldı, uçak mı kalktı ilgilenmezsin.

İşte **Application** ve **Transport** katmanları “Mektubu Gönderen Kişi” gibidir. Veriyi paketler, adresi yazar ve **Network Layer**'a teslim eder. Yolun detaylarıyla ilgilenmezler.

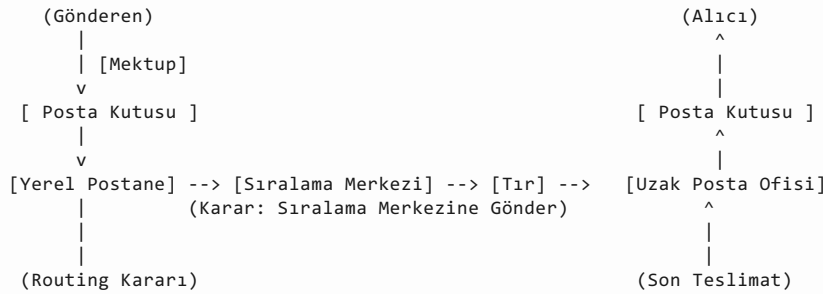
2. Postanenin Gözünden

Senin için bitti ama Posta Servisinin (Network Layer) işi yeni başlıyor.

- Şehir içindeki mektup için belki sadece küçük bir kamyonet yeterlidir.
- Ülkenin ucundaki mektup için: Önce yerel şubeye, sonra ana dağıtım merkezine, oradan uçağa, oradan başka bir kamyona...

Posta servisi her durakta mektuba bakar ve bir karar verir: **“Bunu şimdi nereye göndermeliyim?”**

[Posta Hizmeti Yönlendirme (Routing) Süreci]



Bu zincirdeki her bir durak, ağ dünyasındaki **Router**'dır. Ve bu yönlendirme işlemine **Routing** denir.

Ağ İçin Anlamı

Bu hikayeyi teknik terimlere dökersek:

1. **Addressing = IP Address**: Posta servisi evleri ayırmak için nasıl açık adres (Cadde, Sokak, No) kullanıyorsa; **IP** de her bilgisayara benzersiz bir **IP Address** verir.
2. **Routing = Forwarding Packets**: Postaneler (Routerlar), gelen paketlere bakar ve “Bu

adrese gitmesi için bir sonraki durak neresi?” diye karar verir.

3. **Altyapı:** Posta servisi mektupları taşımak için nasıl kamyonlar, uçaklar ve şubeler kurduysa; **Network Layer** da veriyi taşımak için kablolar, routerlar ve switchlerden oluşan bir altyapı tanımlar.

Özetle: * **Üst Katmanlar (App/Transport):** “Al bunu şuraya gönder, nasıl gittiği umrumda değil.” der. * **Alt Katman (Network/IP):** “Tamam, ben haritaya bakıp en iyi yolu bulacağım ve adım adım götüreceğim.” der.

IP Adreslemenin Temelleri

IP (Internet Protocol), adresleri tanımlarken iki ana kurala dayanır. Bu kurallar kaos çıkmaması için hayati önem taşır:

1. **Benzersizlik:** Ağa bağlanan her cihaz (buna **TCP/IP Host** diyoruz) benzersiz bir adrese sahip olmalıdır. Tıpkı kimlik numaran gibi, kimsede aynı olamaz.
2. **Gruplama:** Adresler rastgele dağıtılmaz, belirli kurallara göre gruplanır.

Posta sistemindeki **ZIP Code** (Posta Kodu) mantığı gibidir. “34 ile başlayanlar İstanbul, 06 ile başlayanlar Ankara” gibi. IP adresleri de sayılarına göre gruplanır.

Dotted-Decimal Notation (Noktalı Ondalık Gösterim)

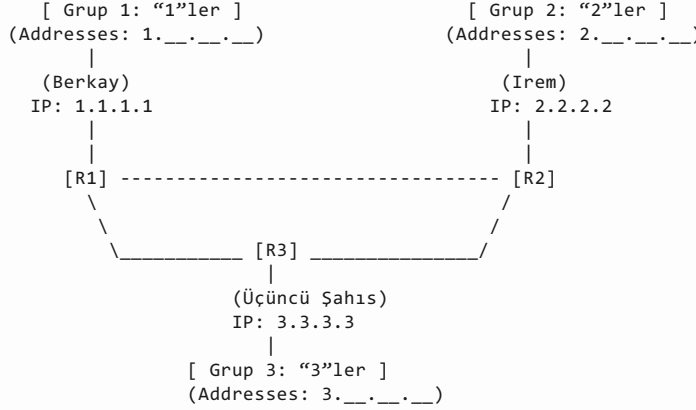
IP adreslerinin yazım şekline havalı bir isim veriyoruz: **Dotted-Decimal Notation (DDN)**.

- **Kural:** Dört adet sayı vardır.
- **Ayırıcı:** Bu sayılar birbirlerinden **nokta** ile ayrılır.
- **Örnek:** 1.1.1.1 veya 192.168.1.1

Mahalle Mantığı

Hadi şimdi canlandıralım. Burada 3 farklı “Mahalle” (Grup) ve bunları birbirine bağlayan “Muhtarlar” (Routerlar) var.

[IP Grupları ile Basit TCP/IP Ağı]



Bu şemada adreslerin başındaki ilk numaraya göre bir gruplama yapıldığını görüyoruz:

- **Sol Taraf (Berkay’ın Mahallesi):** Burada tüm adresler **1** ile başlar (1.__.__.__). Berkay’ın adresi 1.1.1.1 olduğu için buradadır.
- **Sağ Taraf (Irem’in Mahallesi):** Burada tüm adresler **2** ile başlar (2.__.__.__). Irem 2.2.2.2 adresini kullanır.
- **Alt Taraf (Üçüncü Şahıs’ın Mahallesi):** Burada tüm adresler **3** ile başlar (3.__.__.__).

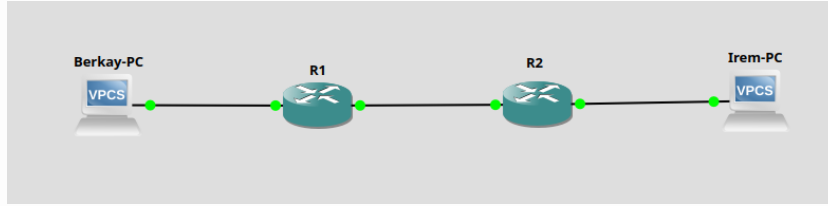
Routerların Rolü

Şemada gördüğün **R1**, **R2** ve **R3**; **Router**'ları temsil eder.

- **Tanım:** Routerlar, TCP/IP ağının parçalarını birbirine bağlayan cihazlardır.
- **Görevi (Routing/Forwarding):** Posta ofisi ne yapıyorsa onu yaparlar.

1. Paketi fiziksel bir **Interface**'den alırlar.
2. Paketin üzerindeki **IP Adresine** bakarlar.
3. Karar verirler: "Himm, bu paket 2 ile başlıyor, o zaman R2'ye göndermeliyim."
4. Paketi doğru yola **Forward** ederler.

Routerlar trafiği yönlendiren polislerdir, IP adresleri de paketlerin üzerindeki "Gideceği Yer" etiketleridir. Gruplama sayesinde Router'lar her tekil evi ezberlemek zorunda kalmaz, sadece "2 ile başlayanlar sağ tarafta" diye bilmeleri yeterlidir.



IP Routing Temelleri

TCP/IP ağ katmanının (Network Layer) temel görevi basittir: **IP Packet**'lerini bir cihazdan diğerine taşımak yani forwarding yapmak. Ağa bağlanan ve bir IP adresi olan her cihaza (boyutu veya gücü ne olursa olsun; süper bilgisayar, telefon veya akıllı buzdolabı) **IP Host** denir.

Paketin İçeriği

Berkay, Irem'e web sayfasının bir parçasını gönderirken, veri paketini katman katman sarar. Artık sadece HTTP ve TCP başlıkları yok, en dışta bir **IP Header** var.

Bu IP Başlığında iki kritik bilgi yazar:

1. **Source IP Address:** Gönderen kim? (Berkay: 1.1.1.1)
2. **Destination IP Address:** Alıcı kim? (Irem: 2.2.2.2)

Adım Adım Yolculuk

Hadi Berkay'ın oluşturduğu bu paketin, Routerlar üzerinden zıplayarak Irem'e gidiş hikayesini izleyelim.

[IP Routing Mantığı - Berkay'dan Irem'e]

Adım 1 (Gönderen)	Adım 2 (Route)	Adım 3 (Alıcı)
-----	-----	-----
Berkay	R1	R2
Irem		
(1.1.1.1)	(Router)	(Router)
(2.2.2.2)		
^		
[IP Packet]	[IP Packet]	
[Src:1.1.1.1]	[Dst: 2.2.2.2]	
[Dst:2.2.2.2]		
+----->	----->	-----+
	(Karar: "2'ye ...")	(Karar: "Local")
	"R2'ye Gönder"	"Doğrudan Gönder"

Adım 1: Berkay Yola Çıkıyor

Berkay paketi hazırladı. Hedef adres 2.2.2.2.

- **Berkay'ın Mantığı:** Berkay, İrem'in tam olarak nerede olduğunu, hangi kablodan gidileceğini bilmez. Tek bildiği şey şudur: **"Bu paketi yakınımıdaki Router'a (R1) atarsam, o gerisini halleder."**

Burası mektubu sokağındaki posta kutusuna atmak gibidir. Mektup kutudan sonra hangi tıra binecek bilmezsin.

Adım 2: R1 Karar Veriyor

Paket **R1**'e gelir. R1 paketi açar ve sadece **Destination IP Address**'e (2.2.2.2) bakar.

- **R1'in Mantığı:** R1 kendi hafızasındaki haritaya (Routing Table) bakar.
- **Karar:** "Hımm, 2 ile başlayan adresler şu tarafta, R2 yönünde." der ve paketi **R2**'ye fırlatır.

Adım 3: R2 Teslim Ediyor

Paket **R2**'ye gelir. R2 de hedef adrese (2.2.2.2) bakar.

- **R2'nin Mantığı:** R2 haritasına bakar ve görür ki 2.2.2.2 (İrem), direkt olarak kendisine bağlı.
- **Karar:** "Bu adres benim yerel ağımda." der ve paketi direkt İrem'in bilgisayarına teslim eder.

IP Routing Lab (Berkay'dan İrem'e Yolculuk)

Hedef: Teoriyi senaryoya canlandırmak. Berkay (1.1.1.1) ve İrem (2.2.2.2) farklı mahallelerde oturuyor. Aradaki R1 ve R2 router'larına yolu öğreterek bunları konuşturacağız.

Adım 1: Topolojiyi Kur

GNS3'te şu düzeni kuruyoruz:

[Topology Diagram]

```

    [Berkay-PC] ----- [R1] ----- [R2] ----- [İrem-PC]
    (1.1.1.1)         (1.1.1.254)       (2.2.2.254)   (2.2.2.2)
                   (WAN: 12.0.0.1)     (WAN: 12.0.0.2)
```

1. Cihazlar:

- 2 adet **VPCS** (Biri Berkay, biri İrem olsun).
- 2 adet **Cisco Router** (Biri R1, biri R2).

2. Kabloleme:

- Berkay -> R1 (FastEthernet0/0)
- R1 (FastEthernet0/1) -> R2 (FastEthernet0/1) [*Burası Routerlar arası yol*]
- R2 (FastEthernet0/0) -> İrem

Adım 2: IP Adreslerini Ver

Burada küçük bir teknik dokunuş yapacağız. Routerlar arası bağlantı için **12.0.0.0** bloğunu kullanacağız (R1 ve R2'nin birleşimi gibi düşün).

A. Bilgisayarlar

Berkay ve İrem'e kimliklerini ve **Gateway** adreslerini verelim.

```
# Berkay PC Konsolu:
# IP: 1.1.1.1, Gateway: 1.1.1.254 (R1'in bacağı)
ip 1.1.1.1 255.255.255.0 1.1.1.254

# Irem PC Konsolu:
# IP: 2.2.2.2, Gateway: 2.2.2.254 (R2'nin bacağı)
ip 2.2.2.2 255.255.255.0 2.2.2.254
```

B. Router R1 Ayarları (Berkay'ın Tarafı)

```
# R1 Konsolu:
enable
conf t

# 1. Bacak: Berkay'a bakan taraf (Local)
interface FastEthernet0/0
ip address 1.1.1.254 255.255.255.0
no shutdown

# 2. Bacak: R2'ye giden taraf (WAN)
interface FastEthernet0/1
ip address 12.0.0.1 255.255.255.0
no shutdown
exit
```

C. Router R2 Ayarları (İrem'in Tarafı)

```
# R2 Konsolu:
enable
conf t

# 1. Bacak: İrem'e bakan taraf (Local)
interface FastEthernet0/0
ip address 2.2.2.254 255.255.255.0
no shutdown

# 2. Bacak: R1'e giden taraf (WAN)
interface FastEthernet0/1
ip address 12.0.0.2 255.255.255.0
no shutdown
exit
```

Adım 3: Routing

Burası labın en kritik kısmı!

Şu an R1, sadece kendine bağlı olanları (1.0.0.0 ve 12.0.0.0) tanır. İrem'in mahallesi olan **2.0.0.0** hakkında hiçbir fikri yoktur. R1'e paketi nereye atacağını öğretmemiz lazım. Yani Postaneye gidip “2 ile başlayan mektupları şu kamyona (R2) yükle” talimatını asıyoruz.

```
# R1 Konsolu (R1'e İrem'in yolunu öğretiyoruz):
# Komut: ip route [Hedef_Ağ] [Maske] [Sıradaki_Durak_IP]
ip route 2.0.0.0 255.0.0.0 12.0.0.2

# R2 Konsolu (R2'ye de Berkay'ın yolunu öğretmeliyiz ki cevap dönebilsin):
ip route 1.0.0.0 255.0.0.0 12.0.0.1
```

Adım 4: Trace

Her şey hazır. İrem'in (PC) Berkay'a bir paket yollayalım.

1. Trace Testi (İz Sürme):

Paketin hangi routerlardan geçtiğini görmek için şu komutu yaz:

```
Irem-PC> trace 1.1.1.1
trace to 1.1.1.1, 8 hops max, press Ctrl+C to stop
 1  2.2.2.254  39.458 ms  9.065 ms  9.554 ms
 2  12.0.0.1   30.243 ms  19.478 ms  19.425 ms
 3  *1.1.1.1   29.526 ms  (ICMP type:3, code:3, Destination port unreachable)
```

Beklenen Çıktı:

1. 1.1.1.254 (Önce R1'e gitti)
2. 12.0.0.2 (R1 onu R2'ye fırlattı)
3. 2.2.2.2 (R2 onu İrem'e teslim etti)

CCNA İçin Neden Önemli?

Bu "Routing" işlemi, CCNA'nın kalbidir. Sınavın yarısından fazlası şu üç soruya odaklanacak:

1. **Addressing:** IP adreslerini nasıl planlarız?
2. **IP Routing:** Paketler yollarını nasıl bulur?
3. **Routers:** Routerlar bu kararları nasıl verir?

Bu temel mantığı (Adrese bak -> Tabloyu kontrol et -> Yönlendir) asla unutma. Tüm internet bu basit döngü üzerine kurulu.

TCP/IP Data-Link ve Physical Layers

IP (Network Layer) paketi adresten adrese götürmeye odaklanır demiştik. Ama IP, fiziksel kablolarla uğraşmaz. "Bu veriyi kablodan nasıl geçiririm?" sorusunun cevabı en alt iki katmandadır.

Ayrılmaz İkili

1. Physical Layer (Fiziksel Katman):

- Tamamen donanım ve fizik.
- **Görevi:** Kabloları ve kabloların içinden geçen enerjiyi (elektrik sinyalleri, ışık vb.) tanımlar.

2. Data-Link Layer (Veri Bağlantı Katmanı):

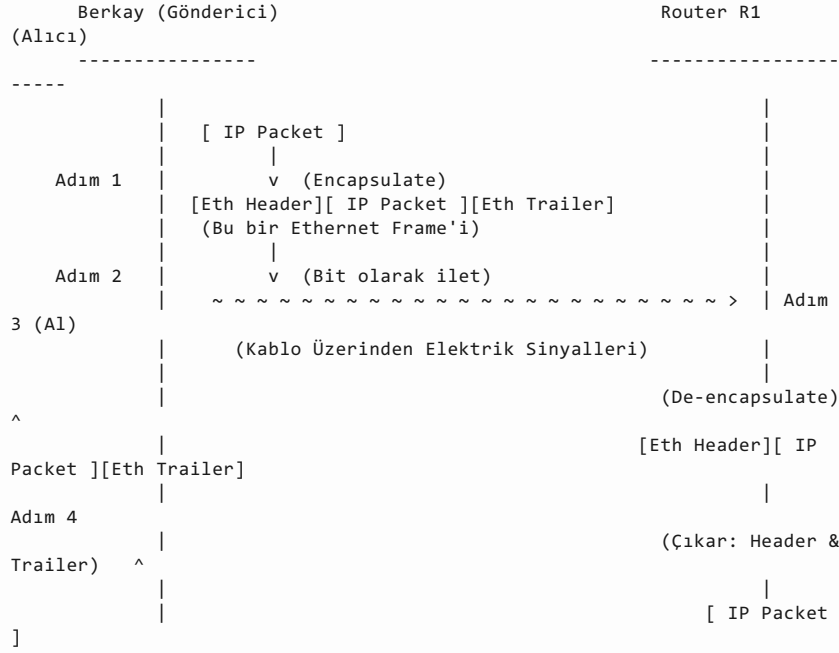
- Kurallar bütünüdür.
- **Görevi:** O kablo üzerinden veriyi göndermek için gereken kuralları ve düzeni sağlar.
- **Hizmet:** Bir üstündeki **Network Layer**'a (IP) hizmet eder.

IP, Berkay'dan R1'e gitmeye karar verir. Ama IP paketi kendi başına kablodan geçemez. Data-Link katmanı devreye girer, IP paketini alır, sarıp sarmalar ve fiziksel yola hazırlar.

Encapsulation & De-encapsulation (Paketleme ve Açma)

Berkay, Router R1'e bir paket göndermek istiyor. Arada bir **Ethernet LAN** var. Bakalım IP paketi nasıl kılık değiştiriyor.

[Berkay Ethernet Kullanarak IP Paketini R1'e İletme]



4 Adımlı Süreç

Bu süreç, ağ iletişiminin temel taşıdır. Adım adım ne olduğuna bakalım:

Adım 1: Encapsulation (Kapsülleme - Berkay)

Berkay'ın IP paketi hazır. Ethernet kartı bu paketi alır ve bir **Zarfın** içine koyar.

- **Header:** Paketin önüne eklenir (**Ethernet Header**).
- **Trailer (Kuyruk):** Paketin arkasına eklenir (**Ethernet Trailer**).
- **Sonuç:** Ortaya çıkan bu yeni yapıya **Ethernet Frame** denir.

Protokoller genelde Header kullanır ama Data-Link katmanında (Ethernet gibi) hem başa **Header** hem de sona **Trailer** eklenir. Tıpkı bir sandviç gibi veriyi araya alırlar.

Adım 2: İletim - Berkay

Berkay, oluşturduğu bu Frame'i alır ve **bit**'lere (0 ve 1) dönüştürür. Sonra bu bitleri kablo üzerinden gelecek **elektrik sinyallerine** çevirip fiziksel olarak gönderir.

Adım 3: Alma - Router R1

R1, kablodan gelen elektrik sinyallerini algılar. Bu sinyalleri tekrar yorumlayarak orijinal **bit**'lere ve dolayısıyla **Ethernet Frame** haline dönüştürür.

Adım 4: De-encapsulation (Zarfı Açma - Router R1)

R1'in asıl ilgilendiği şey zarf değil, içindeki mektuptur (IP Paketi).

- R1, **Ethernet Header** ve **Ethernet Trailer**'ı söktüp atar.
- Geriye tertemiz **IP Packet** kalır.

Özetle: Kim Ne Yapar?

- **Data-Link Layer:** IP paketini alır, bir **Frame** içine hapseder (Encapsulation).
- **Physical Layer:** O Frame'i alır, elektriğe/sinyale çevirip karşıya atar.

Bu işlem sayesinde Berkay ve R1, aradaki kablo ve Ethernet teknolojisini kullanarak paketi bir duraktan diğerine başarıyla taşımış olur.

Data Encapsulation (Veri Kapsülleme Nedir?)

Ağ iletişimde altın kural şudur: **Her katman, bir üstten aldığı veriye kendi imzasını (Header) ekler.**

- **Encapsulation (Kapsülleme):** Verinin etrafına Headers ve bazen Trailers ekleme işlemidir.
- **Amaç:** Veriyi, ağdaki yolculuğu boyunca korumak ve yönlendirmek.

5 Adımlı Süreç

Bir bilgisayarın veri göndermesi, yukarıdan aşağıya doğru gerçekleşen 5 adımlı bir paketleme zinciridir.

[5 Adımda Data Encapsulation]

Layer	Eylem ve Temsili ASCII Art
-	-----
1. Application Layer	(1) Uygulama Verileri Oluşturulur [DATA] v
2. Transport Layer	(2) Transport Layer Header Ekler (TCP/UDP) [TCP DATA] v
3. Network Layer	(3) Network Layer Header Ekler (IP Address) [IP TCP DATA] v
4. Data Link Layer	(4) Data Link Layer Header ve Trailer Ekler (MAC) [Eth Header IP TCP DATA Eth Trailer] v
5. Physical Layer	(5) Physical Layer Bitlere Dönüştürür 10110100101101... (Elektrik Sinyalleri) ->

Adım Adım Neler Oluyor?

1. **Adım 1 (Application):** Uygulama veriyi oluşturur. (Örn: Web sayfası içeriği). Bazen uygulama da kendi HTTP Header'ını ekler.
2. **Adım 2 (Transport):** Veri bir alt kata iner. TCP veya UDP, verinin başına kendi Header'ını yapıştırır. Artık paketimiz "TCP + Data" olmuştur.
3. **Adım 3 (Network):** Paket bir alt kata daha iner. IP, source ve destination adreslerini içeren Header'ını en başa ekler.
4. **Adım 4 (Data Link):** Burası özeldir. Ethernet gibi protokoller veriyi korumak için hem **baş**a (Header) hem de **sona** (Trailer) ekleme yapar. Paket artık tam bir "Ethernet Frame" olmuştur.
5. **Adım 5 (Physical):** Artık etiketleme biter. Bu uzun paket, kablodan geçebilecek sinyallere (**Bits**) dönüştürülür ve yola çıkar.

Dikkat ettiysen sadece **Data Link Layer** (Layer 2) hem Header hem Trailer ekler. Diğer katmanlar sadece Header ekler. Bu fark CCNA sınavlarında sıkça sorulur.

TCP/IP Mesajlarının Adları

Az önce Encapsulation sürecini öğrendik. Şimdi bu sürecin her adımında oluşan o pakete verdiğimiz **özel isimleri** öğreneceğiz. Teknik dilde bunlara **PDU (Protocol Data Unit)** denir ama biz daha yaygın isimlerini kullanacağız.

Ağ uzmanları konuşurken "Şu veriyi yolla" demezler. Hangi katmandan bahsettiklerine göre

şu üç terimi kullanırlar:

1. **Segment:** Transport Layer (Taşıma Katmanı - TCP) mesajı.
2. **Packet:** Network Layer (Ağ Katmanı - IP) mesajı.
3. **Frame:** Data-Link Layer (Veri Bağlantı Katmanı - Ethernet) mesajı.

Burada göreceğin şey şu: Her katman için “Data” kavramı değişiyor.

[Segment, Packet, ve Frame]

Katman & İsim	Yapı
Transport Layer (İsim: SEGMENT)	[TCP Header Data]
Network Layer (İsim: PACKET)	[IP Header Data] (TCP + Uygulama Verilerini içerir)
Data-Link Layer (İsim: FRAME)	[LH] Data [LT] (Link Header)(IP+TCP+Uygulama içerir) (Link Trailer)

Bu arada şemadaki **LH**(Link Header) ve **LT**(Link Trailer), Ethernet gibi protokollerin header ve trailer’ını temsil eder.

“Veri”ye Bakış Açısı

Burada kafanı karıştırabilecek ama çok önemli bir detay var: “Data” kime göre nedir?

- **TCP İçin:** Data = Kullanıcının web sayfası içeriğidir.
- **IP İçin:** Data = TCP Başlığı + Web Sayfası içeriğidir.
- **Ethernet İçin:** Data = IP Başlığı + TCP Başlığı + Web Sayfası içeriğidir.

Şunun gibi bir şey: kargo uçağı (Link Layer) taşıdığı konteynerin (Packet) içinde ne olduğuyla ilgilenmez. İçinde mektup mu var, koli mi var (Segment), yoksa başka bir şey mi var bakmaz. Onun için hepsi “Taşınacak Yük”tür (**Data**). * **Network Layer (IP)** paketi hazırlarken, arkasına taktığı TCP başlığını “TCP Başlığı” olarak görmez, onu sadece taşıması gereken **Data** olarak görür. Her katman sadece kendi eklediği Header’a bakar, gerisini “Data” olarak kabul eder ve bir alt kata paslar.

OSI Ağ Modeli ve Terminolojisi

Bir zamanlar (özellikle 80’lerin sonu, 90’ların başı), herkes geleceğin **OSI Modeli** olacağını sanıyordu. Eğer o senaryo gerçekleşseydi, bugün bilgisayarlarımızda TCP/IP değil, OSI protokolleri çalışıyor olacaktı. Ama ne oldu? **OSI bu savaşı kaybetti**. Bugün dünyadaki hiçbir modern bilgisayar, iletişim kurmak için OSI modelini (protokol seti olarak) kullanmaz.

Peki Neden Öğreniyoruz?

“Madem kullanılmıyor, neden öğreniyoruz?” bunu daha önce not olarak belirtmiştim. Cevap tek kelime: **Terminoloji**. O eski günlerde, herkes OSI kazanacak sandığı için tüm üreticiler (Cisco, HP, IBM) dokümanlarını ve terimlerini OSI modeline göre yazdılar. Bu alışkanlık o kadar yerleşti ki, OSI protokolleri ölse bile **dili** hayatta kaldı. Tıpkı **Latince** gibi düşün. Bugün sokakta kimse Latince konuşmaz (Ölü bir dil). Ama bir doktora gittiğinde sana Latince kelimeler/hastalıklar söyler. Tıp dünyası anlaşılmak için hala Latince terimleri kullanır. Ağ dünyasında da durum aynıdır. Biz **TCP/IP** kullanırız ama birbirimizle anlaşırken **OSI terimlerini** (Layer 1, Layer 7 vb.) kullanırız.

- OSI bir protokol olarak **ÖLDÜ**.
- OSI bir referans modeli ve ortak dil olarak **YAŞIYOR**.

OSI ve TCP/IP'nin karşılaştırılması

Temel mantık olarak OSI ve TCP/IP birbirine çok benzer.

1. İkisinin de katmanları vardır.
2. İkisi de belirli standartlara referans verir (Örneğin **IEEE Ethernet** standartları ikisi için de ortaktır, Amerika'yı yeniden keşfetmezler :D).

Ancak katmanların sayısı ve isimlendirilmesinde farklılıklar vardır. Bugün OSI modelini, diğer modelleri kıyaslamak için bir **referans cetveli** olarak kullanıyoruz.

Aşağıda sol tarafta 7 katmanlı OSI, sağ tarafta ise bugün kullandığımız modern 5 katmanlı TCP/IP var.

[OSI vs. TCP/IP]

OSI Model (7 Layers)		TCP/IP Model (5 Layers)
7. Application \		
6. Presentation >	(Birleşir) ---->	5. Application
5. Session /		
4. Transport	----->	4. Transport
3. Network	----->	3. Network
2. Data Link	----->	2. Data Link
1. Physical	----->	1. Physical

1. Alt 4 Katman:

- Dikkat ettiysen, alt katmanlar (1, 2, 3 ve 4) her iki modelde de **isim, numara ve işlev** olarak birebir aynıdır.
- Physical, Data Link, Network, Transport. Burada kafa karışıklığı yok.

2. Üst Katmanlar:

- Fark tepede başlıyor. OSI bu kısmı çok detaylandırıp **Session, Presentation** ve **Application** diye 3'e böler.
- TCP/IP ise daha pratik davranır: "Bunların hepsi uygulamayla ilgili işler" der ve tek bir **Application** katmanında toplar.

Neden Katman 7 Diyoruz?

İşte burası sahadaki mühendislik jargonu için kritik nokta. Dünya TCP/IP kullanıyor dedik. TCP/IP modelinde (sağdaki) en üst katman **5. Katmandır**. Ama bir ağ uzmanına gidip "HTTP bir Layer 5 protokolüdür" dersin sana garip garip bakar.

Kural: Biz protokolleri numaralandırırken hala **OSI numaralarını** kullanırız.

- **HTTP:** TCP/IP'de en tepededir ama biz ona "**Layer 7 Protokolü**" deriz.
- **Router:** Network katmanındadır, "**Layer 3 Cihazı**" deriz.
- **Switch:** Data Link katmanındadır, "**Layer 2 Cihazı**" deriz.

TCP/IP'de Application katmanı, OSI'nin 5, 6 ve 7. katmanlarının (Session, Presentation, App) yaptığı tüm işleri kapsar. Ama biz konuşurken alışkanlıktan dolayı hep "**Layer 7**" terimini kullanırız.

OSI Data Encapsulation Terminoloji (PDU Nedir?)

Hatırlarsan TCP/IP dünyasında mesajlara katmanına göre özel isimler veriyorduk:

- Layer 4: **Segment**
- Layer 3: **Packet**
- Layer 2: **Frame**

OSI modeli ise bu özel isimlerle uğraşmaz. Hepsine tek bir genel isim verir: **Protocol Data Unit (PDU)** - (Protokol Veri Birimi).

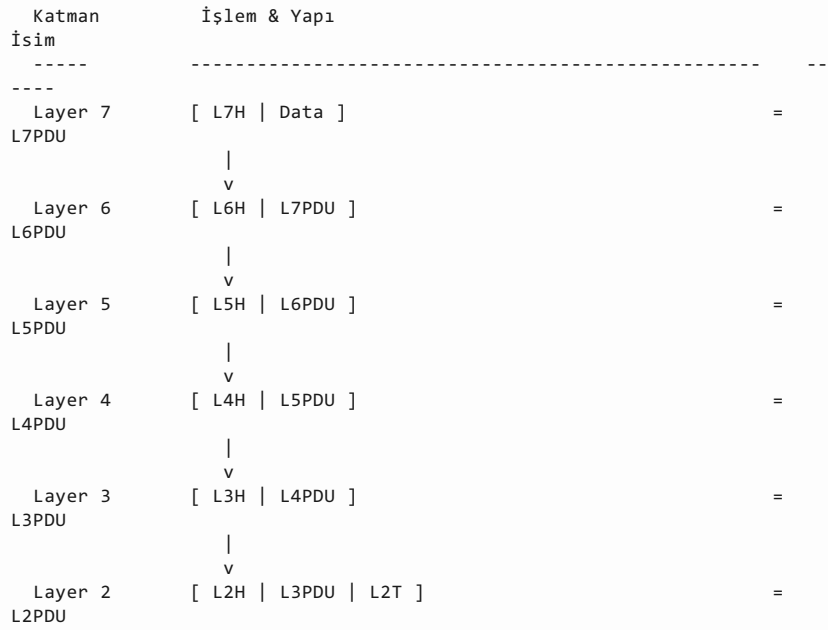
OSI'nin isimlendirme mantığı çok basittir: **Layer X PDU (LxPDU)**. Buradaki “x”, katmanın numarasını temsil eder.

- **Layer 7 verisi:** L7PDU
- **Layer 4 verisi:** L4PDU
- **Layer 3 verisi:** L3PDU

TCP/IP’de “IP Packet” dediğimiz şeye, OSI dilinde **L3PDU** (Layer 3 PDU) deriz. Çünkü IP bir 3. Katman protokolüdür.

Veri en tepeden (L7) başlar ve aşağı indikçe her katman kendi başlığını (**H- Header**) ekler. En altta (L2) ise hem header hem de trailer eklenir.

[OSI Encapsulation ve PDUs]



- **L#H:** Layer # Header (Katman Başlığı)
- **L#T:** Layer # Trailer (Katman Kuyruğu - Sadece Layer 2’de var!)

Sınavda kafan karışmasın diye şu tabloyu bir yere ekle:

Layer Numarası	TCP/IP Name (Özel İsim)	OSI Name (Genel İsim)
Layer 4	Segment	L4PDU
Layer 3	Packet	L3PDU
Layer 2	Frame	L2PDU
Layer 1	Bit	L1PDU